CUDA Kernels - Getting Started Notes

Why CUDA is part of this project

An LLM inference engine spends most of its time doing large amounts of numerical work on model weights and intermediate values. A CPU is very good at running a small number of complicated tasks quickly. A GPU is built to run a huge number of simpler tasks at once.

That does not mean a GPU magically makes everything faster. The work has to be split into lots of pieces that can run independently. CUDA is NVIDIA's programming model and toolchain for writing that GPU work from C++.

In this engine, C++ does the host-side work:

- load and parse model files
- tokenize prompts
- allocate and manage memory
- decide what operation runs next
- launch GPU work

CUDA kernels do the device-side work:

- gather embedding values
- normalize activation vectors
- run matrix operations
- apply attention
- sample or select the next token

The basic setup is not "replace C++ with CUDA." It is one program with two sides:

```
CPU / host: organize work and launch operations
GPU / device: execute lots of numerical work in parallel
```

The first thing to keep straight: host vs device

Host memory is normal CPU RAM. Device memory is GPU VRAM. They are separate memory spaces.

```
CPU code can directly access host memory.
GPU kernels can directly access device memory.
```

So this is not valid as a general assumption:

```
"I have a C++ vector, so the GPU kernel can read it."
```

It cannot. First allocate GPU memory, then copy the data over.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

```
int *token_ids_gpu = nullptr;

cudaMalloc(&token_ids_gpu, count * sizeof(int));

cudaMemcpy(
    token_ids_gpu,
    token_ids_cpu,
    count * sizeof(int),
    cudaMemcpyHostToDevice);
```

The same idea applies to outputs. A kernel can write output into GPU memory. We only copy that output back to CPU memory if the CPU actually needs it. In inference engines, keeping data on the GPU between operations is important. Copying activations back and forth after every layer would destroy performance.

Think of CPU RAM and GPU VRAM as two workshops with a transport lane between them. Moving a small list of token IDs is fine. Moving a whole room of model weights repeatedly is not. Bring the model into the GPU workshop once, then keep using it there.

What a kernel actually is

A CUDA kernel is a C++-like function that executes on the GPU.

```
__global__ void addKernel(const float *a, const float *b, float *out, int count)
{
    const int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index >= count)
    {
        return;
    }

    out[index] = a[index] + b[index];
}
```

`__global__` says this function is launched by the host and runs on the device.

The code looks like it runs once, but when we launch it, the GPU runs many copies of it. Every copy is a separate GPU thread with its own `threadIdx` and therefore its own `index`.

If `count` is one million and we launch at least one million threads, then thread 0 adds item 0, thread 1 adds item 1, and so on.

This way of thinking is more useful than thinking "a kernel runs on every GPU core." You write a thread program. CUDA and the GPU scheduler take care of running a huge number of those threads across the actual hardware.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

CUDA's hierarchy: thread, block, grid

CUDA organizes a launch into three levels:

```
grid
  block 0
    thread 0
    thread 1
    ...
  block 1
    thread 0
    thread 1
    ...
```

- A **thread** is one execution of the kernel.
- A **block** is a group of threads that can cooperate with each other.
- A **grid** is every block in one kernel launch.

For a one-dimensional array, this is the standard global index calculation:

```
const int index = blockIdx.x * blockDim.x + threadIdx.x;
```

Example launch:

```
3 blocks
4 threads per block
```

Block	Thread	Global index
0	0	0
0	1	1
0	2	2
0	3	3
1	0	4
1	1	5
1	2	6
1	3	7
2	0	8
2	1	9
2	2	10
2	3	11

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

`blockIdx.x` tells a thread which group it belongs to. `threadIdx.x` tells it where it is inside that group. `blockDim.x` tells it how many threads are in each block.

CUDA also supports `x`, `y`, and `z` dimensions. We can use a 2D or 3D layout when the data naturally fits that shape. But a 1D flattened index is often simpler and is exactly what the embedding gather kernel uses.

Launching a kernel

Host code launches a kernel with CUDA's triple-angle-bracket syntax:

```
addKernel<<<blocks, threads_per_block>>>(a_gpu, b_gpu, out_gpu, count);
```

The first value is how many blocks to launch. The second is how many threads are in each block.

For a one-thread-per-element kernel, calculate the number of blocks with ceiling division:

```
const int threads_per_block = 256;
const int blocks =
    (count + threads_per_block - 1) / threads_per_block;

addKernel<<<blocks, threads_per_block>>>(a_gpu, b_gpu, out_gpu, count);
```

Why round up? We have to launch enough threads for the final element even if the length is not a multiple of 256.

```
count = 1,000
threads per block = 256
blocks = (1000 + 255) / 256 = 4
threads launched = 4 * 256 = 1,024
```

The final 24 threads have no useful element to process. This is normal. It means the kernel needs a bounds check.

Bounds checks are not optional

Every simple elementwise CUDA kernel should usually have this near the top:

```
if (index >= count)
{
    return;
}
```

This lets unused threads in the final block exit safely.

Without it, a thread could write to `out[1003]` when an output array only has valid indices `0` through `999`. GPU out-of-bounds access can give wrong output, an illegal-memory-access error, or something that looks fine until a later unrelated kernel fails. That makes it annoying to debug.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The common kernel pattern

Most first CUDA kernels follow this structure:

```
__global__ void kernelName(
    const InputType *input,
    OutputType *output,
    int count)
{
    const int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index >= count)
    {
        return;
    }

    output[index] = someOperation(input[index]);
}
```

Then a host launcher calculates the grid size and launches it:

```
cudaError_t launchKernel(
    const InputType *input,
    OutputType *output,
    int count)
{
    const int threads_per_block = 256;
    const int blocks =
        (count + threads_per_block - 1) / threads_per_block;

    kernelName<<<blocks, threads_per_block>>>(input, output, count);
    return cudaGetLastError();
}
```

This is not the only way to write CUDA, but it is a good mental starting point. First make the work mapping obvious. Then verify it is correct. Then think about performance.

Why keep a launcher separate from the kernel

We could write `kernel<<<...>>>(...)` directly in `main.cpp`, but a launcher is cleaner for an inference engine.

- The rest of the program calls a normal C++ function such as `launchEmbeddingGather`.
- The launch geometry lives next to the kernel implementation.
- We get one place to add launch-error handling.
- We can change the block size, add a CUDA stream, or tune the launch later without changing every caller.

The launcher is the boundary between CPU orchestration and GPU computation.

```
host code
-> launcher decides how many GPU threads are needed
-> kernel runs those threads
-> GPU output stays in VRAM for the next kernel
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Threads do not always mean one element

One thread per output element is a good first mapping, but it is not a rule.

For embedding gather, one thread writes one BF16 output value. That works because every value is independent.

For RMSNorm, one output vector needs a sum across all 2,048 values. Threads need to collaborate, so the mapping changes: one block handles one token and every thread works on two values. The threads then combine partial sums through a reduction.

For matrix multiplication, a thread may calculate one output value, several values, or part of a tile. Libraries like cuBLAS have heavily tuned implementations for this, which is why we will use them for the big matrix multiplications rather than assume a basic custom kernel will beat them.

The question before writing a kernel is always:

```
What is the smallest independent piece of output work,
and what data does that thread need to produce it?
```

Global memory, shared memory, and registers

There are different places a GPU thread can keep data.

Global memory

Global memory is the big VRAM pool. Kernel arguments like `embed_tokens`, `token_id_gpu`, and `activations_gpu` point into global memory.

- Large capacity.
- Visible to all threads and kernels.
- Much slower than on-chip storage.

Model weights and large activation tensors live here because they are too large for anything else.

Registers

Registers are tiny, very fast storage private to one thread.

```
const int index = ...;
const float x = input[index];
```

Values like `index` and `x` will commonly live in registers when possible. Other threads cannot directly read a thread's registers.

Shared memory

Shared memory is small, fast storage shared by threads in the same block.

```
__shared__ float partial[1024];
```

It is useful when threads in a block need to cooperate, like calculating a sum, max, or reduction. RMSNorm uses it for partial squared sums.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Shared memory is not shared across the whole grid. A thread in block 0 cannot use shared memory from block 1. This block-level boundary is one of the main things to understand before writing reductions and attention kernels.

Synchronization

Threads in a block do not run one after another in a predictable order. If one thread writes shared memory and another thread immediately reads it, we need a barrier to make sure all relevant writes are done first.

```
__syncthreads();
```

This means: every thread in this block must reach this point before any can continue past it.

Think of it like a group project checkpoint. Nobody starts using the shared whiteboard results until everyone has written their assigned part.

`__syncthreads()` only works for threads in the same block. There is no simple normal barrier for every thread in an entire grid during one standard kernel launch. If blocks must communicate, we normally use multiple kernel launches or a different algorithm.

One important rule: do not put `__syncthreads()` inside a branch that only some threads take. Every thread in the block must reach it, otherwise the block can hang.

Warps: the group the GPU actually schedules together

CUDA gives us threads and blocks, but the GPU hardware typically executes threads in groups of 32 called warps.

```
one block of 256 threads
= 8 warps of 32 threads
```

The threads in a warp normally execute the same instruction together. If an `if` statement sends some warp threads one way and other threads another way, the GPU usually has to execute both paths while masking off the threads that are not active for that path. This is called warp divergence.

Example:

```
if (index % 2 == 0)
{
    doA();
}
else
{
    doB();
}
```

This is not automatically wrong. It is just work to be aware of when optimizing. The bounds check in a final partial warp or final partial block is normal and usually worth the simplicity.

For now, the useful rule is: nearby thread indices often run together, so give nearby threads similar work and nearby memory accesses when you can.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Memory access patterns and coalescing

GPUs are fast when neighboring threads access neighboring addresses in global memory. Hardware can combine those reads or writes into fewer memory transactions. This is called coalescing.

Good pattern:

```
out[index] = input[index];
```

If threads 0 through 31 run together, they access 32 adjacent values.

Less friendly pattern:

```
out[index] = input[index * 2048];
```

Now neighboring threads jump far apart through memory.

Embedding gather is partly irregular because different prompt positions may select totally different vocabulary rows. But inside one selected row, it is good: adjacent threads load adjacent hidden values. In this engine, 256 threads per block divides the 2,048 hidden size exactly, so a block handles a contiguous section of one embedding row.

This is a useful distinction: we cannot always make every access perfect, but we can make the contiguous part of the work simple and GPU-friendly.

CUDA is asynchronous

When the CPU calls:

```
embeddingGatherKernel<<<blocks, threads_per_block>>>(...);
```

the call usually queues GPU work and returns before the GPU has finished. This lets the CPU keep organizing later work instead of waiting around.

There are two different error moments to understand:

```
return cudaGetLastError();
```

This checks whether launching the kernel failed immediately, for example because the launch configuration was invalid.

```
cudaDeviceSynchronize();
```

This waits for earlier GPU work to complete. It can surface errors that happened while the GPU was actually executing the kernel, such as an illegal memory access.

During early development, synchronization makes debugging much easier because an error is reported close to the kernel that caused it. In a more optimized engine, we avoid unnecessary synchronization because making the CPU wait kills overlap and throughput.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

A minimum debugging checklist

When a first kernel gives wrong results, check these before trying to do clever optimization:

1. Is every input pointer device memory, not a CPU pointer?
2. Did the `cudaMalloc` byte count use `element_count * sizeof(type)`?
3. Is the host-to-device or device-to-host direction correct in `cudaMemcpy`?
4. Does every thread calculate a unique valid index?
5. Is there a bounds check for rounded-up launches?
6. Is the logical matrix shape consistent with the flattened index calculation?
7. Do input and output use the intended dtype, for example `int` IDs and BF16 activations?
8. Did we call `cudaGetLastError()` after the launch?
9. When debugging, did we synchronize before trusting the result?
10. Can we compare a tiny test case against a CPU or PyTorch reference result?

Small tests are way easier to reason about. Before a 128,256 by 2,048 table, make a fake table with four rows and four columns. Then manually verify every output value.

Things to avoid early on

- Do not optimize before checking correctness against a reference.
- Do not use CPU pointers inside a kernel.
- Do not forget that CUDA launches are asynchronous.
- Do not assume every model has a 2,048 hidden size. This project hard-codes it because it targets one Llama model.
- Do not assume all GPU threads can synchronize with all other threads. Synchronization and shared memory are block-local.
- Do not use BF16 blindly for every intermediate calculation. Numerical operations such as sums and reductions often need FP32 accumulation even when the input and output are BF16.
- Do not write a custom matrix multiplication just because it is possible. Start with cuBLAS for the big GEMMs and only write custom code when there is a specific reason.

How this grows into an inference engine

The embedding gather is the easy first kernel: independent reads and writes.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The next operations introduce the more interesting GPU concepts:

```
Embedding gather
  one thread per output value
  no thread cooperation

RMSNorm
  one block per token
  shared memory and reduction

Q, K, V and MLP projections
  large matrix multiplications
  cuBLAS GEMMs

RoPE, residual adds, SiLU, softmax
  custom elementwise or reduction-style kernels

Attention
  careful memory layout, reduction, causal masking, and eventually KV cache access
```

The CUDA basics do not change. Every operation still needs us to decide:

1. What data is on the GPU?
2. What output are we producing?
3. How do we split that output into thread-sized pieces?
4. Do threads need to communicate?
5. What memory pattern will those threads create?
6. How do we validate the result before optimizing it?

That is really the low-level job of an inference engine. The model gives us the weights and the math. CUDA lets us decide how that math is mapped onto the GPU.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.